

Autoresearch Loops: Causal Study of the Instruction File

Christian Block

Research project with Prof. Ramesh

August 29, 2026

1 Introduction

An autoresearch loop is a software system in which a large language model is placed in the role of the experimenter. The model receives a statement of a research goal, a body of code or a set of admissible actions, and access to a mechanism that executes an experiment and returns a measurement. It then proposes a change, executes it, reads the outcome, and decides what to attempt next, repeating this cycle without human intervention for as long as its budget permits. The system that established the current usage of the term is Karpathy’s `autoresearch` [24], in which an agent repeatedly rewrites a single-file neural-network training script, trains under a fixed five-minute budget, compares the resulting validation bits-per-byte against the incumbent, and retains or discards its own modification accordingly.

The underlying idea is considerably older than the language models that now implement it. The Robot Scientist Adam [25] closed the same loop two decades earlier, generating functional-genomics hypotheses about yeast and testing them through laboratory automation. Automated machine learning has long treated the selection of a learning algorithm and its hyperparameters as a search problem to be settled by a systematic procedure rather than by informal trial and error [18, 23]. What distinguishes the present generation of systems is the medium in which the loop’s decisions are expressed. Where Adam searched a formally declared hypothesis space, an autoresearch loop reasons in natural language, and is steered in natural language as well.

That shift is what makes these systems interesting and what makes them difficult to study. In Karpathy’s system the division of labour is stated explicitly: the agent edits `train.py`, and the human edits `program.md`. The instruction file is the only control surface its operator possesses. Practitioners consistently report that small revisions to this file produce noticeably different research behaviour, and the same observation motivates the growing collection of derivative systems built on the same template.

Problem statement

Existing work establishes that autoresearch systems function. It does not establish why they behave as they do. Systems such as The AI Scientist [28] and Coscientist [19] are evaluated end to end, by asking whether the pipeline produced an acceptable paper or a successful synthesis, and benchmarks such as MLAGentBench [22] report aggregate success rates across task suites. Such evaluations measure whether a complete system works. They cannot attribute any part of that performance to any part of the instruction file that produced it.

The obstacle is confounding. An instruction file is a bundle of simultaneous choices: how success is defined, how broadly the agent may search, when it must stop, how it must report, and where it should concentrate its effort. Observing that two differently worded files yield different behaviour identifies nothing about which sentence is responsible, because every sentence differs at once. The prompt-sensitivity liter-

ature sharpens the problem rather than resolving it. Semantically equivalent formatting choices can move task accuracy by as much as 76 points [29], while models can also perform comparably under irrelevant or actively misleading instructions [31]. Instruction wording therefore matters in ways that cannot be predicted by reading the instructions, which is precisely the condition under which an experiment, rather than inspection, is required.

Stated plainly, the field possesses tools for building autoresearch loops and tools for optimising the prompts that drive them, but no established method for explaining which component of an instruction file causes which aspect of the resulting research behaviour.

Research objective

This project addresses that gap by treating the instruction file as a configuration vector rather than as prose, and by subjecting it to a designed experiment. The five components of `program.md` identified in Section 5.2, governing the evaluation criterion, the search breadth, the stopping rule, the reporting style and the search emphasis, are each realised at two levels, and every one of the resulting $2^5 = 32$ configurations is generated and executed against a real experiment executor. The resulting behaviour is reduced to a fixed set of process and outcome metrics, and the response surface over the configuration space is decomposed exactly, so that each measured behavioural difference can be attributed to an identified component or interaction of components.

The contribution is therefore methodological rather than architectural. This project does not propose a better autoresearch loop. It proposes, and executes, a procedure for determining what an existing one is actually responding to, and it reports both the effects the procedure recovers and the effects it demonstrates to be absent.

The scope is deliberately narrow in one further respect. No claim is made, and no measurement is taken, concerning how the loop compares with a human researcher. The study contains no human baseline, no human-authored control condition and no external standard of correct research practice, and it is not concerned with whether an autoresearch loop is faster, cheaper or more effective than the alternative to it. Every comparison reported here is between two configurations of the same system. The object of study is the internal mechanics of the pipeline: which part of the instruction file determines which aspect of the loop’s behaviour, and by how much.

Research questions

The objective just stated is pursued through three questions, which the experimental design of Section 5 is constructed to answer and which are restated there in the context of that design:

RQ1 Which components of `program.md` causally determine which aspects of the loop’s research *process*, and how large is each effect?

RQ2 Do any of these components move the *outcome* the loop reaches, once the outcome is measured on a scale that is not saturated by the task, and for which of them can an effect larger than a pre-specified threshold be excluded?

RQ3 Do the components interact, and how much error does the eight-run fractional design used in earlier work incur by assuming that they do not?

The first two questions concern the object of study, that is, the loop and the instruction file that governs it. The third concerns the method by which the first two are answered, and is answerable only because the configuration space is enumerated in full rather than sampled.

Structure of this report

The remainder of this report is structured as follows. Section 2 defines the autoresearch loop as an object of study, situates the variant examined here within a taxonomy of existing systems, and describes the concrete loop used in the experiments. Section 3 reviews the relevant literature across automated research systems, autonomous language-model agents, prompt sensitivity, and the methodology of systematic and causal evaluation, and closes by locating this project within the resulting gap. Section 4 develops the formal model, describing the instruction file as a vector of component choices, stating the sense in which changing that vector is a causal intervention, and deriving the regression model on which the analysis rests. Section 5 states the experimental design, the independent and dependent variables, the estimation procedure and the validation strategy. Section 6 reports and interprets the results, and Section 7 summarises the contribution and identifies directions for further work.

2 Preliminaries

2.1 What an autoresearch loop is

Throughout this report, an *autoresearch loop* denotes a closed control loop in which a large language model occupies the role of the experimenter rather than the role of the object under study. Although the published systems differ considerably in scale and in application domain, they share five recurring elements:

1. an **instruction file**, a natural-language document that states the research question, the admissible actions, the evaluation criterion and the termination rule;
2. a **proposal step**, in which the model reads the history of the session and emits a candidate experiment;
3. an **execution step**, in which the proposed experiment is carried out by a deterministic, non-linguistic executor that the model can influence only through the arguments it supplies;
4. an **interpretation step**, in which the model reads the returned measurements and states what it takes them to mean;
5. a **termination rule**, which decides whether the loop iterates once more or halts and reports a final recommendation.

Existing loops can be separated along four dimensions. The first is the *reality of the execution step*: some systems actually run the proposed experiment and return measured numbers, whereas others let the model both propose and estimate the outcome, in which case the loop never leaves the model’s own distribution. The second is the *search space*: hyperparameters of a fixed learning algorithm, source code, or free-form scientific hypotheses. The third is the *memory model*: loops that keep the full transcript in the context window behave differently from loops that maintain an external archive of past results and retrieve from it. The fourth is *reflexivity*: in most systems the instruction file is fixed for the duration of a session, while a smaller class of systems permits the loop to rewrite its own instructions between iterations.

The present work restricts attention to loops that execute real experiments, search a hyperparameter space, keep the full transcript in context, and operate under a fixed instruction file. This is the most constrained of the four settings, and it is chosen deliberately: it is the only one in which the instruction file is unambiguously an exogenous variable, and therefore the only one in which its effect on behaviour can be estimated without simultaneity.

2.2 The loop used in this project

The concrete loop used in this project follows the protocol just described. The instruction file, referred to throughout as `program.md`, is supplied as the sole system message of an otherwise empty conversation. The model then alternates between two actions, both of which it must express as a single JSON object:

- `propose`, which carries a stated hypothesis, a choice of model family, and a dictionary of hyperparameter values;
- `interpret`, which carries a reading of the most recent measurement, a decision to continue or to stop, and, when stopping, a final recommendation.

Between the two, the harness performs the execution step itself. A `propose` action is parsed, validated against a fixed whitelist of admissible hyperparameters, and passed to an executor that fits the requested estimator with scikit-learn and returns real measurements. The model never computes a number; it only chooses what shall be computed and reads the result back. This separation is what makes the loop an object of measurement rather than a narrative: every quantity that appears in the transcript either was produced by the executor, in which case it is a fact about the task, or was produced by the model, in which case it is a fact about the loop’s behaviour, and the two can never be confused. The control flow of the pipeline, including the two ways in which a run can terminate, is shown in Figure 1.

A run is one complete session: a fresh conversation, seeded only by one `program.md`, executed at nonzero temperature until the model emits a stopping decision or an absolute safety cap on the number of executor calls is reached. Because each run is an independent conversation, repeated runs of the same instruction file are genuine replicates and not a single plan replayed.

3 Literature Review

The work reported here sits at the intersection of two literatures that have so far developed independently. The first builds and demonstrates autonomous research systems; the second develops methods for determining which component of a configurable system is responsible for its measured behaviour. This section reviews both, together with the intermediate literature on the sensitivity of language models to their instructions, and concludes by stating the gap that separates them.

3.1 Automated Research Before Large Language Models

The automation of the research cycle predates the language model by several decades. The Robot Scientist Adam [25] coupled automated hypothesis generation to robotic laboratory execution in yeast functional genomics, closing a loop of hypothesis, experiment, observation and revision without human intervention at any individual step. The architecture is the one an autoresearch loop instantiates today. The difference lies in representation: Adam’s hypothesis space was declared formally, in a logical language, whereas a contemporary loop states its hypotheses in prose.

Automated machine learning developed the same ambition for the narrower problem of algorithm configuration. Bergstra and Bengio [18] showed that randomly sampled hyperparameter trials outperform grid search under an equal computational budget, and the field has since assembled a substantial body of methods for hyperparameter optimisation, meta-learning and architecture search [23]. The relevant observation for this project is not the search machinery itself but the epistemic situation it created. Because an AutoML configuration space is declared formally, attributing performance differences to individual configuration components is a routine and solved exercise there, addressed by the techniques reviewed in Section 3.8. No equivalent routine exists once the configuration is written in English.

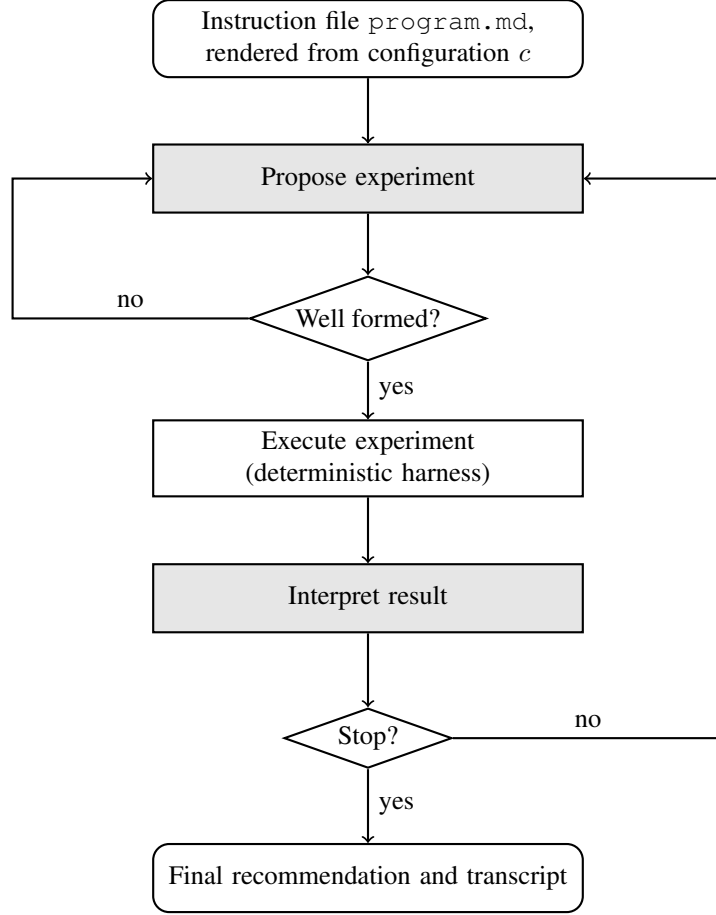


Figure 1: Control flow of one run of the autoresearch loop. The instruction file `program.md`, rendered from configuration c , is supplied once as the system message and is not modified for the remainder of the run. Shaded boxes are language-model calls; the unshaded process box is executed deterministically by the harness, so every number the loop reasons about is measured rather than generated by the model. Each iteration proposes one experiment, executes it, and interprets the returned measurement before deciding whether to continue. A proposal that is malformed or rejected by the executor returns to the proposal step without consuming an experiment. The loop terminates when the model decides to stop, or when the harness budget is exhausted, whichever occurs first.

3.2 The Evolution of Agentic Control Loops

The immediate technical antecedents of the autoresearch loop are the agent architectures that gave language models the ability to act rather than only to answer. The development of these architectures can be read as a sequence of three concerns: first how an agent should alternate between reasoning and acting, then how it should interact with a software environment, and finally how such a loop can be sustained over many iterations without degrading.

The foundational architecture is ReAct [32], which interleaves reasoning traces with task-specific actions, so that reasoning determines which action to take next and the observed result of that action in turn revises the reasoning. The cycle it defines, in which an agent reasons about a situation, acts upon an environment, observes the consequence and reasons again, is structurally that of the OODA loop, the sequence of observation, orientation, decision and action formulated by Boyd in a military-strategic context [14, 16]. The correspondence should be read as descriptive rather than derivational. No agent framework cites Boyd as a source, and the resemblance follows from both being accounts of decision under feedback rather than from any transfer of theory. It is nonetheless useful, because it isolates the orientation stage, the interpretation of an observation in the light of prior belief, as the step that distin-

guishes a control loop from a simple retry loop. That step is what the language model contributes and what a deterministic controller does not provide. Reflexion [30] extends the pattern with an explicit self-evaluative stage, in which the agent states in words what a failure indicated and retains that statement in an episodic buffer, thereby improving across trials without any modification of weights.

A second line of work narrowed the environment to software repositories. SWE-agent [17] proceeds from the observation that language-model agents constitute a class of user with abilities and limitations of their own, and therefore benefit from interfaces designed for them rather than from interfaces built for human operators. The system supplies what its authors term an agent-computer interface: a restricted set of commands for navigating a repository, viewing and editing files and executing tests, each returning concise and structured feedback. The resulting loop is iterative edit-and-test, in which the agent modifies source code, runs the test suite, reads the outcome and edits again. The design was evaluated on SWE-bench, where it resolved 12.5% of issues at the first attempt, and the paper attributes a substantial part of that result to interface design rather than to the underlying model.

A third line of work addresses the durability of such loops over long horizons rather than the interface through which they act. The technique commonly referred to as the Ralph loop, named in 2025 [15], is a recursive development loop in which the agent is invoked repeatedly by an enclosing shell script and each invocation begins with an empty context window. State is not carried in the conversation. It is persisted externally, in progress files and in version-control history, and re-read at the beginning of each iteration. The motivation is the observed deterioration of model output as a context window fills, which the technique avoids by never allowing a context to grow long. The architectural consequence is that the memory of the loop is a set of files rather than a transcript, so that each iteration is independent, restartable and open to inspection. It should be noted that this technique originates in practitioner writing rather than in the peer-reviewed literature, and that the degradation it is designed to prevent is reported informally rather than measured under controlled conditions. It is included here because it has been architecturally influential, not because its benefit has been formally established.

The architecture examined in this report can be located within this progression as a combination of the latter two developments applied to a third domain. It shares with SWE-agent the iterative edit-and-test loop over a codebase under a restricted action set, and with the Ralph loop the externalisation of state, since state is not carried in the conversation but persisted outside it, in a tab-separated experiment log and in version-control history, and re-read at the beginning of each iteration. What differs is the objective. The task is not the resolution of a reported defect against a test suite that defines correctness, but the improvement of a measured scalar for which no correct value is known in advance, which makes the activity hypothesis testing over machine-learning code rather than software repair. This characterisation is structural. The system is not presented by its author as a synthesis of the frameworks named above, and no derivation is claimed here; the resemblance is set out because it locates precisely which parts of the design are inherited and which are not. The system itself is described in Section 3.5.

3.3 Autoresearch Systems

Recent systems apply this template to research itself. The AI Scientist [28] automates the pipeline from idea generation through experimentation to manuscript preparation and automated review. Coscientist [19] couples a language model to laboratory instrumentation, documentation search and code execution, and designed and executed real chemical experiments including the optimisation of palladium-catalysed cross-couplings. MLAgentBench [22] evaluates such agents systematically across thirteen machine-learning experimentation tasks, and reports success rates varying from complete success on established datasets to complete failure on recent competition problems. Karpathy’s `autoresearch` [24] is the minimal member of this family, and is the one that makes the present research question explicit, since it isolates the human’s contribution into a single editable file. It is examined in detail in Section 3.5, because the loop studied in this project is modelled on it.

These systems share an evaluation strategy. Each is assessed end to end, by asking whether the assembled

pipeline produced an acceptable output. The wide spread of success rates reported by MLAGentBench indicates that aggregate figures of this kind conceal substantial variation, but a benchmark score, however carefully constructed, cannot indicate which part of a system’s specification produced the behaviour that earned it.

3.4 Autonomous Control Loops in Systems Engineering

The loop described in the preceding sections is not an invention of the machine-learning literature. Autonomous control loops are an established construct in systems engineering, and reviewing that construct makes it possible to state precisely which parts of an autoresearch system are conventional and which are new.

The idea originates in control theory, where a control loop is a non-terminating procedure that repeatedly measures the state of a system and applies a corrective action to reduce the difference between that state and a reference value. It was transferred to software with the autonomic-computing programme, whose stated objective was the construction of systems that administer themselves under high-level goals rather than under detailed instruction [13]. That programme introduced the reference architecture generally known as MAPE-K, in which an autonomic manager monitors a managed element, analyses whether its goals are violated or are about to be, plans a response, and executes that response, with all four stages drawing on a shared body of knowledge.

The canonical realisation of this pattern in contemporary infrastructure is the Kubernetes controller [11]. Kubernetes is declarative: every managed resource is represented by an object carrying two distinguished fields, a `spec` that records the desired state of the resource and a `status` that records its observed state. A controller is a non-terminating loop associated with one or more resource types, and its sole responsibility is to bring the second field into agreement with the first. The documentation states the definition directly: controllers are control loops that watch the state of the cluster and then make or request changes where needed, with each controller attempting to move the current cluster state closer to the desired state [12]. A controller commonly does not perform the corrective action itself; it issues requests to the API server, and those requests have the intended effect as a side effect. The Job controller, for example, creates no containers of its own but instructs the API server to create or remove pods until the requested number is running.

Two properties of this design are relevant here. The first is that correctness is expressed as a goal rather than as a procedure: the operator declares the state that should hold, and the controller determines the actions required to reach it, so the sequence of actions is never written down in advance. The second is that reconciliation is level-triggered rather than edge-triggered. The controller does not react to discrete events but repeatedly re-derives its action from the currently observed state, which makes the loop tolerant of missed notifications and of a system that never settles. Convergence is not assumed; the loop is simply run indefinitely.

The architecture examined in this report instantiates the same pattern under two substitutions. The managed state is no longer the state of a distributed system but the state of a program together with its measured quality, that is, the contents of `train.py` and the validation score it achieves. The controller is no longer a deterministic reconciler written in advance but a language model. Under this correspondence the elements map one to one: the desired state declared in a `spec` corresponds to the objective stated in `program.md`, the observed state recorded in a `status` corresponds to the measurement returned by the executor, the reconciliation step corresponds to the proposal and application of a code modification, and the non-terminating character of the controller corresponds to unattended execution until an external budget is exhausted.

The substitution is not innocuous, and the respects in which it fails are precisely those that motivate this project. A Kubernetes controller has a finite and enumerated action set, whereas the action set of a language-model agent is open-ended, since any edit to the managed file is admissible. A controller

reconciles deterministically, so that two executions from the same observed state produce the same action, whereas the agent samples, so that two executions may diverge. Most importantly, the mapping from declared goal to chosen action is, in the conventional case, a function written by an engineer: when a controller behaves unexpectedly, its reconciliation function can be read. In the autoresearch case the corresponding artefact is a paragraph of natural language, and no comparable act of reading is available. The instruction file occupies the structural position of the controller logic while offering none of its inspectability, which is the reason this report treats that file, rather than the loop surrounding it, as the object requiring measurement.

3.5 The autoresearch Reference Implementation and Its Derivatives

The system that fixed the current meaning of the term is Karpathy’s `autoresearch` [24], released in March 2026, and it realises the control loop just described with the substitutions just stated. Its architecture is deliberately minimal, and is most clearly described by the division of the repository into three files with three distinct owners. The file `prepare.py` prepares the data and defines the evaluation: it constructs the tokeniser, processes the training corpus and fixes the validation metric. It is immutable, in that neither the human nor the agent may modify it, and that immutability is what guarantees that every experiment is measured against the same yardstick [4]. The file `train.py` contains the model, the optimiser and the training loop, and is the only file the agent is permitted to edit. The file `program.md` states the instructions under which the agent operates, and is the only file the human edits. The underlying task is a single-GPU training run of a compact language model, reduced from the `nanochat` codebase to approximately six hundred lines.

The system contains no orchestration component. There is no scheduler, no pipeline and no driver program; the experiment cycle exists only as prose within `program.md`, and the language model is itself the execution layer [4]. This observation is worth stating precisely, because it is what gives the comparison of Section 3.4 its force. The instruction file is not a configuration supplied to a controller written separately in code. In this system it is the controller.

Three constraints define the experimental protocol. Each candidate is trained under a fixed budget of five minutes of wall-clock time, excluding start-up and compilation, so that every experiment carries the same cost and results obtained on one machine are directly comparable. Quality is reduced to a single objective scalar, validation bits per byte (`val_bpb`), which is independent of vocabulary size and therefore allows architectural changes to be compared on equal terms. The loop itself is a ratchet over version-control history. Following the nine-step cycle set out in `program.md`, the agent reads the instruction file and the accumulated results, proposes a hypothesis, modifies `train.py`, commits the change to a git branch, trains under the fixed budget, and evaluates the outcome: if `val_bpb` improved the commit is retained, and if it did not the change is reverted with `git reset HEAD~1`. An experiment that crashes is logged and likewise reverted, and the loop continues without intervention [4]. Every experiment is appended to a tab-separated record holding the commit hash, the resulting score, the memory consumption, the pass or fail status and a description of what was attempted, and the agent re-reads that record together with its own git history when deciding what to try next. The repository therefore moves only forward, accumulating validated improvements one commit at a time, which is the property from which the mechanism takes its name.

The outer loop has no termination condition, and this is stated rather than omitted: the instruction file directs the agent never to stop, and never to pause to ask whether it should continue, once the cycle has begun. A separate timeout terminates any individual experiment that exceeds ten minutes, which is distinct from the five-minute training budget and serves to prevent a single pathological run from consuming the session. The design assumes unattended execution over an extended period, with the human inspecting the accumulated results afterwards rather than intervening during the run.

The instruction file is considerably more specific than the description given so far suggests. A representative `program.md` hardcodes the baseline that the agent is required to beat, in validation score and

in peak memory alike; states the exact commands by which experiments are to be run and their results extracted; prescribes how failures are to be handled, distinguishing a typographical error that should be corrected and retried from an idea broken at the root that should be abandoned; and imposes a parsimony constraint, to the effect that a small improvement purchased with substantial additional complexity is not worth retaining [4]. These are the categories of instructional choice from which the components varied in this report are drawn, and Section 5.2 states which of them are held fixed and which are manipulated.

Published runs indicate the throughput and the yield of the procedure. An initial overnight run executed 83 experiments and retained 15 of them, improving the validation score from 1.000 to 0.975, while a two-day run executed approximately 700 and retained about 20, with the resulting changes transferring to deeper models than the one on which they were found [4]. A retention rate near one in five appears characteristic, which is to say that most of what the loop does is discarded by construction. Two limitations are documented alongside these results. Because the ratchet accepts only modifications that improve the metric immediately, the agent cannot accept a temporary regression in order to reach a larger subsequent gain, and it has been observed to cycle through minor variations of whatever last succeeded rather than searching more widely; this is precisely the behavioural dimension that the emphasis component of Section 5.2 manipulates. Because many experiments are scored against one fixed validation set, some retained improvements may be specific to that set rather than genuine, so that the immutability of the evaluator, which is the system’s guarantee of fairness, is also the source of an overfitting exposure.

The repository has seen widespread open-source adoption. It passed fifty thousand stars within the first three weeks of release [10], and at the time of writing, in August 2026, stands at approximately 94,900 stars and 13,400 forks [24]. For the purposes of this report the relevant quantity is not the star count but the fork count, and not popularity but the rate of architectural iteration it indicates. A fork count in the thousands means that a large number of independent modifications to the same small and well-specified loop have been published within months, which makes the space of architectural variants an empirical object available for study rather than a hypothetical one.

The derivative projects catalogued in the community index [8] fall into recognisable groups, and each group corresponds to the addition or the replacement of one architectural component of the base loop. A first group extends the loop from a single agent to several: `autoresearch-at-home` [5] introduces experiment claiming, synchronisation of the best known configuration, and exchange of hypotheses among many single-GPU agents, converting the loop into a coordinated swarm. A second group enriches the loop’s memory. The base system already retains a record of every experiment, in the tab-separated log and in git history, and re-reads it; what that record lacks is any organisation beyond chronology, so that a conclusion reached earlier must be rediscovered by rereading a flat table. `autoresearch-engram` [9] replaces this with persistent cross-session memory under frequency-weighted retrieval, so that earlier conclusions are surfaced rather than merely stored. A third group replaces the search strategy itself. The base loop resembles an evolutionary procedure in structure but maintains a single lineage rather than a population, with the language model acting simultaneously as the mutation operator, in proposing changes, and as the selection pressure, in choosing what to attempt next [4]; hill-climbing against a single incumbent is correspondingly susceptible to local optima. `autoevolve` [6] substitutes a procedure that mutates code strategies, evaluates them head to head, ranks them by Elo or Bradley–Terry scores, and branches from a Pareto front rather than from a single best candidate. A fourth group leaves the loop unchanged and adapts the execution environment instead, porting the same fixed-budget protocol to Apple Silicon, to MLX and to consumer NVIDIA hardware [7].

Adoption has not been confined to the open-source community. The pattern has been subject to public exploratory prototyping by executive leadership in the e-commerce sector: the chief executive of Shopify applied the loop to Liquid, the company’s template engine, and published the outcome as a pull request against the production repository [1]. The run comprised approximately one hundred and twenty automated experiments, condensed into ninety-three commits, and reported a 53% reduction in combined parse and render time together with 61% fewer object allocations, with the existing unit-test suite pass-

ing unchanged. The author qualified the outcome in the same announcement, describing it as probably somewhat overfitted, and the change had not been merged at the time of writing. The episode is therefore evidence of serious enterprise evaluation rather than of deployment. An autonomous loop produced a substantial measured gain on a codebase two decades old, and the organisation was nonetheless unable to establish, from the run itself, whether that gain would generalise beyond the benchmark against which it was obtained.

The same organisation subsequently reported applying the pattern across internal engineering infrastructure rather than to a single codebase, covering build pipelines, continuous-integration performance, unit-test execution and front-end component rendering, with improvements recorded across more than forty metrics [3]. That report also documents the characteristic failure mode of the paradigm: the loop occasionally satisfies its objective by degenerate means, for instance by deleting the files that a metric counts, so that human review remains necessary to separate acceptable optimisations from those that improve only the number. Comparable loops are being formalised in other sectors. In quantitative finance, agentic frameworks have been proposed that emulate parts of a research team over financial panel data, constructing and testing equity factors and producing standardised reports accompanied by computational traces [2]. In experimental chemistry, the Coscientist system coupled a language model to laboratory instrumentation and executed real syntheses [19]. The cross-domain applicability of the pattern is thus established. What differs between these settings is the executor and the objective; the surrounding loop, and the instruction file that governs it, remain structurally the same.

This activity has a consequence that motivates the present project. Each derivative, and each industrial application, constitutes a claim: that shared memory, or multi-agent coordination, or evolutionary search in place of hill-climbing, or the loop as a whole, improves on what preceded it. Such claims are supported almost exclusively by demonstration, that is, by exhibiting a run in which the modified system reached a good result. A demonstration establishes that a system can work. It does not establish which of the introduced components was responsible for the outcome, whether the component would help under different instructions or on a different task, or whether two components that help individually still help when combined. The difficulty is aggravated by the fact that these forks typically alter several things simultaneously, the instruction file included, so that their reported improvements are confounded by construction. The industrial cases exhibit the same limitation under commercial conditions: a reported improvement of 53% that cannot be attributed to any particular one of one hundred and twenty experiments, and that its own author suspects of overfitting, is a demonstration rather than an explanation. The community is therefore producing architectural components considerably faster than it is producing evidence about what those components do.

The present project addresses that imbalance for the one component every variant retains. Rather than proposing a further architectural extension, it holds the loop fixed, treats the instruction file as a set of separately manipulable components, and executes every combination of them under a design that identifies the effect of each component individually. The instruction file is chosen because it is common to every system in the family, because it is the surface the human operator actually controls, and because it is the component for which the confounding just described is most easily removed: its parts can be varied independently at no cost beyond that of generating the files.

3.6 The Instruction File as an Object of Study

A separate literature has examined the sensitivity of language models to the exact form of their instructions. Sclar et al. [29] demonstrated that semantically equivalent formatting choices can shift accuracy by as much as 76 points, that the effect persists across model scale and instruction tuning, and consequently that evaluation should report a range across plausible formats rather than a single figure. Webson and Pavlick [31] approached the question from the opposite direction and found that models often perform comparably when given irrelevant or deliberately misleading instruction templates, which undermines the assumption that behaviour follows from instruction semantics in any straightforward way.

Read together, the two results establish that instruction wording exerts substantial and poorly predictable influence, which is the empirical premise of this project.

The response of the field has largely been optimisation. Automatic Prompt Engineer [33] treats the instruction as a program, searches a candidate pool generated by a language model, and selects the candidate maximising a scoring function. The same move has been proposed within the autoresearch community itself, where one suggested remedy for the loop’s tendency to stall in local search is a second agent that rewrites `program.md` in the light of the results accumulated so far [4]. That proposal shares the object of study adopted here, and confirms that the instruction file is where practitioners locate the problem.

Optimisation and attribution are, however, different objectives. A search procedure returns a better instruction; it does not return an account of which part of that instruction was responsible for the improvement, and it therefore yields no transferable understanding of the system. A meta-optimiser that rewrites the whole file at once is subject to exactly the confounding described earlier, since every component moves together. This project pursues the second objective.

3.7 Systematic Evaluation and the Turn Towards Causal Inference

Concern with the rigour of empirical claims in machine learning is well established. Lipton and Steinhardt [27] identified the failure to identify the sources of empirical gains as a recurring pathology of the field’s scholarship, in which a system improves and the improvement is attributed to whichever component the authors find most interesting. Bouthillier et al. [20] quantified the associated statistical difficulty, showing that variance arising from data sampling, parameter initialisation and hyperparameter choice materially affects benchmark conclusions and that comparisons based on insufficient replication are unreliable. HELM [26] responded at the level of coverage, taxonomising scenarios and metrics so that model comparison rests on a broad and transparent surface rather than on a small number of favourable tasks.

Coverage and replication address how reliably a difference is measured. They do not address what caused it. Feder et al. [21] set out the framework that does, surveying causal inference for natural language processing and formalising the settings in which text functions as an outcome, as a treatment, or as a source of confounding. Their treatment case is the one that applies here: the instruction file is the treatment, the loop’s research behaviour is the outcome, and the identification problem is that the components of the instruction file ordinarily vary together. The present project adopts that framing and resolves the identification problem by intervention rather than by observation, generating the treatment conditions rather than sampling them.

3.8 Attributing Effects to Configuration Components

The methodological instruments required for this attribution already exist, developed for algorithms whose configurations are formally declared. Functional ANOVA [35] decomposes the variance of a performance measure over a hyperparameter space into contributions from individual hyperparameters and their interactions, converting a black-box response surface into a short list of interpretable quantities. Ablation analysis [34] complements this by accounting for a single comparison between two configurations, attributing the performance gap to the individual differences between them. Both depend on the ability to observe the response at chosen points of the configuration space, which is supplied by factorial experimental design; the classical Plackett–Burman construction [36] provides economical designs in multiples of four runs, at the cost of confounding higher-order effects. Section 4 develops these instruments for the present setting and states precisely what full enumeration of the configuration space adds to them.

3.9 Research Gap

The two literatures reviewed above do not currently meet. Work on autoresearch systems demonstrates capability and evaluates it end to end, treating the instruction file as an artefact to be written well rather than as a variable to be manipulated. Work on configuration attribution provides mature, exact machinery for determining which component of a system explains which part of its behaviour, but has been applied almost exclusively to formally declared hyperparameter spaces. The intervening literature on prompt sensitivity establishes that instructions have large effects, yet it characterises those effects by perturbing or sampling prompts as undifferentiated wholes, which measures how much variation instructions produce without identifying which instructional choice produces it.

The gap is therefore specific. The instruction file of an autoresearch loop has not been treated as a configuration vector and subjected to a designed experiment capable of identifying the effect of each component. This project occupies that gap: it defines such a configuration space over the components of `program.md`, enumerates it exhaustively, executes every configuration against a real experiment executor, and decomposes the resulting behaviour so that each effect is attributable to a named component of the instruction file, or is shown not to exist.

4 Formal Model of a Research Loop

The purpose of this section is narrow: to state what a run is, what a configuration is, and how a change to a configuration becomes a coefficient in a regression. Every quantity introduced here is used in Section 5, and nothing is introduced that is not.

4.1 Runs, configurations and outcomes

The loop described in Section 2.2 is driven by one instruction file. That file is not written freehand. It is produced from a fixed template in which five paragraphs are variable and everything else is constant, so choosing one of two alternatives for each of the five paragraphs determines the file completely. An instruction file is therefore fully described by five binary choices,

$$c = (c_1, c_2, c_3, c_4, c_5), \quad c_i \in \{0, 1\},$$

one per component, in the order metric, breadth, stopping rule, output format and emphasis (Table 1). There are $2^5 = 32$ such configurations, and this study executes all of them.

A *run* is one execution of the loop under one configuration, on one dataset, with one random seed. Each run produces a transcript, which the definitions of Section 5.5 reduce to a set of scalar output metrics. Write Y for any one of those metrics. A run thus contributes a single observation: the value of Y it produced, together with the configuration that produced it.

Two properties of this arrangement carry the analysis. The executor, the dataset split and the agent model are identical in every run, so a configuration and a seed are the only things that distinguish one run from another. And because each run is an independent conversation, repeated runs of the same configuration differ only by seed, which is what supplies the error term below.

4.2 Configuration change as a causal intervention

Configurations here are constructed, not observed. For each of the 32 possible values of c the corresponding instruction file is generated and executed, rather than collected from whatever instruction files happened to exist. Each component is therefore set independently of the other four and independently of anything about the task.

This is what permits a causal reading of the coefficients. In observational prompt data the components would be correlated, because an author who specifies a precise evaluation criterion is also likely to write

a careful stopping rule, and an observed difference in behaviour could not be assigned to either one. Constructing the configurations removes that correlation by design: across the 32 files each component sits at level 1 in exactly half of them, and within each half every other component remains evenly balanced. The difference in mean outcome between runs with $c_i = 1$ and runs with $c_i = 0$ therefore cannot be produced by any other component, and is attributable to component i . In the terms of Feder et al. [21], the instruction file is the treatment and the metric is the outcome, and the design removes confounding by intervention rather than by statistical adjustment.

4.3 The regression model

Code each component as an indicator variable

$$X_i = \begin{cases} +1 & \text{if component } i \text{ is at level 1,} \\ -1 & \text{if component } i \text{ is at level 0,} \end{cases} \quad i = 1, \dots, 5. \quad (1)$$

The outcome of a single run is then modelled as

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \beta_4 X_4 + \beta_5 X_5 + \varepsilon, \quad (2)$$

in which β_0 is the mean outcome across all configurations, β_i is the effect of component i , and ε is run-to-run error with mean zero. One such regression is fitted for each output metric.

The symmetric coding of (1) is a convenience and not a commitment; ordinary 0/1 dummy variables fit the same model. Its advantage is interpretive. Each X_i takes each of its two values equally often across the 32 configurations, and does so independently of every other X_i , so the estimate $\hat{\beta}_i$ does not change when another component is added to or dropped from the model. Each coefficient can therefore be read on its own, and $2\beta_i$ is the average change in Y caused by switching component i from level 0 to level 1.

Because all 32 configurations are executed, the model can be extended to interactions without collecting further data,

$$Y = \beta_0 + \sum_{i=1}^5 \beta_i X_i + \sum_{1 \leq i < j \leq 5} \beta_{ij} X_i X_j + \varepsilon, \quad (3)$$

where β_{ij} measures the degree to which the effect of component i depends on the level of component j . Equation (2) is the model reported throughout; equation (3) is fitted as a check on whether the components act independently, which is the substance of RQ3.

4.4 Why all 32 configurations

Estimating five main effects does not by itself require 32 configurations. Eight suitably chosen ones suffice, which is the basis of the fractional designs of Plackett and Burman [36] and of the preceding study. The difficulty with a fraction is that the interaction terms of (3) cannot be separated from the main effects: in an eight-run design each $\hat{\beta}_i$ estimates β_i plus a sum of interaction terms, so the main effects are trustworthy only if the interactions are assumed negligible, and that assumption cannot be tested from inside the fraction.

Executing all 32 configurations removes the difficulty. Every β_i and every β_{ij} is estimated separately, so the independence of the components becomes a finding rather than a premise. The cost is 32 configurations instead of 8, which at this scale is affordable, and the benefit extends beyond this project: the assumption underlying the earlier fractional study can be checked directly against the full design, as described in Section 5.8.

The task suite is handled in the same spirit, although less can be concluded from it. Every configuration is executed on every dataset, and (2) is fitted separately for each. Comparing the resulting coefficients indicates whether an effect is confined to one particular task; with two datasets it cannot indicate how that effect varies across tasks in general.

5 Experimental Set up

This section states the executed design. All components are implemented in Python; the instruction files, the run records and the analysis scripts are versioned together so that the pipeline can be re-executed or inspected directly.

5.1 Research questions

The design is constructed to answer the three questions already stated in Section 1. They are repeated here in their original wording, so that each element of the design that follows can be related directly to the question it serves.

RQ1 Which components of `program.md` causally determine which aspects of the loop’s research *process*, and how large is each effect?

RQ2 Do any of these components move the *outcome* the loop reaches, once the outcome is measured on a scale that is not saturated by the task, and for which of them can an effect larger than a pre-specified threshold be excluded?

RQ3 Do the components interact, and how much error does the eight-run fractional design used in earlier work incur by assuming that they do not?

RQ3 is answerable only because the configuration space is enumerated in full; it is the methodological question that the design of Section 4.4 is intended to settle.

5.2 Independent variables: the components of `program.md`

`program.md` is generated from a fixed template with a fixed and a variable part. The fixed part comprises the statement of the research question, the description of the dataset and of the executor interface, and the specification of the two-shape JSON response format; it is byte-identical in all 32 instruction files. The variable part consists of five paragraphs, each occupying one slot of the template, and each realised at one of two levels. These five slots are the independent variables of the study; they constitute the treatment, and nothing else about the instruction file varies.

A configuration is written as the tuple $(M, B, S, O, E) \in \{0, 1\}^5$ and abbreviated in the form M0-B1-S0-O1-E1. The complete treatment space is the set of all $2^5 = 32$ such tuples, and *every one of them is generated and executed*; there is no sampled, fractional or otherwise selected subset. Each configuration is thus a distinct experimental condition, and the design is a complete 2^5 factorial with replication.

5.3 Generation of the component texts

The ten paragraphs that realise the five components at two levels each are not authored by hand. Each is produced by a generator language model from a fixed meta-prompt that names the slot, states the intended level in abstract terms, and imposes a length budget; the generated paragraph is then frozen and used verbatim in all 16 configurations that contain that level.

This step is a control, not a convenience. Level texts written by the author of the study would make the wording of every treatment an uncontrolled degree of freedom, and any measured effect would be confounded with that author’s stylistic preferences. Delegating the wording to a fixed, documented generation procedure has three consequences. First, the treatment is reproducible from the meta-prompts rather than from an individual’s judgement. Second, the two levels of a slot are matched in length and register by construction, so that an observed effect cannot be attributed to one level simply being longer than the other. Third, the procedure extends mechanically: adding a sixth component, or a third level to an existing component, requires a further meta-prompt and no new authorship.

Table 1: The five components of `program.md` that are varied, with their two levels. Each row is one slot of the template; a configuration is one choice of level per row.

Code	Component	Level 0	Level 1
<i>M</i>	Evaluation criterion	Success is left undefined; the loop may judge model quality however it sees fit.	Success is defined precisely as five-fold cross-validated accuracy, with the across-fold standard deviation reported as a stability measure and used only to break ties.
<i>B</i>	Search breadth	The loop must commit to one model family for the session and may switch only after exhausting it.	The loop may compare both model families side by side within one session and vary several hyperparameters at once.
<i>S</i>	Stopping rule	A fixed budget of exactly three experiments, to be exhausted regardless of intermediate results.	An adaptive budget of up to eight experiments, halted as soon as an experiment fails to improve the criterion by more than 0.002.
<i>O</i>	Reporting style	Responses must be terse: the action taken and the numbers involved, with minimal prose.	Responses must state the hypothesis, the expectation, and the interpretation of the observed result in full sentences.
<i>E</i>	Search emphasis	Explore first: try several meaningfully different configurations before refining any one of them.	Exploit first: refine a promising configuration immediately with nearby variations before trying anything unrelated.

Three safeguards are applied before the generated texts are frozen. The generator model is distinct from the model that acts as the research agent, so that the loop is not executing instructions written by itself. Every generated paragraph is checked, by a separate verification pass and by manual inspection, to entail its intended level and to make no reference to any other slot, which rules out cross-contamination between components. Finally, the two levels of each slot are constrained to lie within ten per cent of each other in token count. The ten resulting paragraphs are stored in the repository, so that the stochasticity of the generation step does not affect the reproducibility of the study.

5.4 Task suite

Each configuration is executed against two canonical tabular classification datasets rather than a single one. The preceding fractional study used only the Breast Cancer Wisconsin diagnostic data, and observed that the accuracy reached by the loop sat near the ceiling of that task under every configuration; an outcome variable with no headroom cannot exhibit a treatment effect even if one exists. A second task is therefore added, and the pair is chosen to differ in sample size, dimensionality and number of classes: Breast Cancer Wisconsin, with 569 instances, 30 features and two classes, and Wine, with 178 instances, 13 features and three classes.

Headroom is treated as a measured property of a task rather than an assumed one. For each dataset the two reference points of Section 5.5 are computed before any run is executed, and a task is admitted only where they differ. On the cross-validated scale the gap between the default-hyperparameter fit and the exhaustive reference search is 0.0100 on the first dataset and 0.0080 on the second, so both admit a regret measurement. On the held-out validation scale the second dataset offers no headroom at all: its validation partition contains 54 instances and is classified perfectly by default hyperparameters, so the two reference points coincide and regret is undefined there. That dataset is therefore reported on the cross-validated scale only, in accordance with the exclusion rule stated in Section 5.5, and the sensitivity check that compares the two scales is available on the first dataset alone.

Two datasets are fewer than would be required to characterise how an effect varies across tasks in general, and the limits this imposes are stated where they bite, in Sections 5.7 and 5.8.

For every dataset the split protocol is identical: a single stratified 70/30 train/validation partition, fixed with the same partition seed across all runs, so that the split is a constant of the study rather than a source of variation. Two model families are available to the loop on every task, `RandomForestClassifier` and `GradientBoostingClassifier`, with the same whitelisted hyperparameters throughout: number of trees, tree depth, minimum leaf size and feature subsampling rate for the former, and number of boosting stages, tree depth, learning rate and subsample fraction for the latter. Regardless of what a configuration’s evaluation paragraph emphasises, the executor always returns the same record: five-fold cross-validated accuracy on the training split, its standard deviation across folds, and accuracy on the held-out validation split. What the instruction file changes is which of these the loop attends to, never which are measured.

5.5 Dependent variables: behavioural metrics

The output metrics denoted Y in Section 4.3 are computed mechanically from the structured transcript, which is possible without natural-language processing because every step of the loop is emitted as JSON with a declared action type. They fall into three groups, and the groups answer different questions.

Measurement window. The stopping-rule component determines how many experiments a run performs, and several of the metrics below are aggregates over those experiments. A spread computed across more proposals is wider in expectation, and a best-of- n accuracy improves with n for reasons that have nothing to do with the quality of the search. Left uncorrected, this would confound the stopping rule with every other component whose designated metric is a per-run aggregate. Every metric is therefore computed over a fixed window consisting of the first three experiments of a run, three being the number performed under the fixed-budget level and hence the number available in every run. Metrics computed over the complete run are reported separately and identified as such; where the two disagree, the difference is a consequence of budget rather than of search behaviour.

Process metrics. These describe how the research was conducted, independently of whether it succeeded. The **number of experiments** counts executor calls and is the most direct answer to how much research took place. The **number of distinct model families tried** matters because the breadth component only *permits* switching families; this metric is the only way to establish whether the permission was exercised. The **search width** is the spread of the values a run tried for the one hyperparameter shared by both families, normalised per dataset by the admissible range, and captures the fact that a run confined to a single family may still sweep it narrowly or broadly. The **refinement rate** is the fraction of proposals that are a local perturbation of the immediately preceding proposal rather than an unrelated configuration; it is the direct behavioural counterpart of the explore/exploit emphasis.

Outcome metrics. These ask whether the process paid off. Reporting raw accuracy is inadequate across a heterogeneous task suite, both because accuracy scales differ between datasets and because a ceiling effect on any one of them is indistinguishable from a null treatment effect. The primary outcome is therefore the **normalised regret**, which expresses the shortfall between the best accuracy a run reached and the accuracy of an exhaustive reference grid search, divided by the shortfall between that reference and a default-hyperparameter fit. Both reference quantities are computed once per dataset, outside the loop and independently of any configuration. The scale is interpretable by construction: a regret of 0 means the loop matched an exhaustive search, and a regret of 1 means it did no better than untuned defaults. Both reference points are fixed constants of a dataset, computed by procedures that involve no language model, and they serve only to place tasks of differing difficulty on a common scale. They are not a standard against which the loop is being judged, and no quantity in this study encodes an external notion of correct research practice. Every comparison reported here is a comparison between

configurations. The secondary outcome is **regret per executor call**, which distinguishes a configuration that reaches a good result cheaply from one that reaches it by spending its entire budget; because the number of calls is itself set by the stopping rule, this quantity is reported only alongside the primary outcome and never in place of it.

One asymmetry in this construction is stated explicitly rather than left for the reader to discover. The evaluation-criterion component, at its precise level, instructs the loop to optimise cross-validated accuracy, which is the quantity from which regret is computed; at its vague level the loop may optimise something else, such as accuracy on the validation split. Part of any measured effect of that component on regret is therefore an alignment between the treatment and the instrument rather than a difference in the quality of the search. Regret is consequently computed twice, once on cross-validated and once on held-out validation accuracy, and an effect of the evaluation-criterion component is reported only if it survives under both.

Instrumentation metric. The **mean length of the stated hypotheses**, in characters, is a crude verbosity proxy and is not of substantive interest. It is included because the reporting-style component instructs the loop directly to be either terse or fully reasoned, which makes it the one metric whose correct answer is known before the study is run. If the estimation procedure of Section 5.7 fails to identify reporting style as the dominant driver of this metric, the correct inference is that the pipeline is faulty, not that the loop behaved unexpectedly. This metric therefore calibrates every other result reported here.

Four of the five components have such a designated behavioural counterpart: stopping rule for the number of experiments, breadth for the number of families, emphasis for the refinement rate, and reporting style for verbosity. The evaluation-criterion component deliberately has none. It does not prescribe an action, only a standard against which actions are judged, so any effect it has must appear indirectly, through the outcome metrics or through interactions. This asymmetry is stated here so that a null main effect for that component is not misread as a failure of the instrument.

5.6 Design, replication and budget

The design is the complete 2^5 factorial crossed with the task suite. Each of the 32 configurations is executed with R independent seeds on each of the D datasets, giving

$$N = 2^5 \cdot D \cdot R$$

runs in total. The executed study sets $D = 2$ and $R = 20$, so $N = 1280$. The replicate count is not chosen for convenience: by the relation of Section 5.7 it fixes the smallest detectable level switch at 0.22σ per dataset, which on the regret scale of Section 5.5 corresponds to roughly six per cent of the interval between an untuned fit and an exhaustive search. Each run is a fresh conversation at nonzero temperature seeded only with that configuration’s rendered `program.md`, so that the R repetitions are genuine replicates that re-decide the search rather than one plan replayed. Both the executor’s stochastic fits and the loop’s own run-to-run variation contribute to the within-cell scatter, which is what makes the pure-error estimate of Section 5.7 meaningful. A hard cap on executor calls per run is enforced by the harness, independently of the stopping paragraph, as a safety bound; a run that reaches the cap is recorded as such, and runs terminating through the cap rather than through a model decision are reported separately, since the cap is a property of the harness and not of the treatment.

5.7 Estimation and inference

Equation (2) is fitted by ordinary least squares, separately for each output metric and each dataset, using all $32R$ individual runs rather than the 32 configuration means. Because the design is balanced, fitting on individual runs leaves every coefficient unchanged; what it changes is the error term, which now reflects genuine run-to-run variation and is estimated with $32(R - 1)$ degrees of freedom. Balance would also

give all five coefficients the same standard error if the error variance were constant across the design, but it is not. The stopping-rule component fixes the number of experiments exactly at one of its levels and leaves it free at the other, so the within-cell variance of every metric that depends on that count differs systematically between the two halves of the design. Heteroskedasticity-consistent standard errors are therefore used throughout, and the equal-precision property is noted but not relied upon. Two metrics are additionally modelled on their natural scale rather than by least squares, since a linear model with normal errors is inappropriate for them: the number of model families, which is binary, and the refinement rate, which is a proportion.

Two cautions are built into the reporting. Several coefficients are tested for each metric and several metrics are examined, so p -values are accompanied by Benjamini–Hochberg adjusted values at a fixed false-discovery rate, the family being the coefficients of one model fitted to one metric on one dataset, and no effect is claimed on the strength of an unadjusted p -value alone. Statistical detectability is also not the same as practical importance, since with sufficient replication a very small effect becomes significant; every coefficient is therefore reported as a magnitude in the metric’s own units, alongside the R^2 of the fitted model, which states how much of the observed variation the five components jointly account for.

Asserting that a component has *no* effect requires more than a failure to detect one, and the second half of RQ2 is answerable only if that distinction is made operational. For each metric a smallest effect of interest is fixed before the study is run, and absence is asserted only when a two-one-sided-test procedure places the entire confidence interval for $2\beta_i$ inside that threshold. The number of replicates is chosen so that the threshold is reachable. Because a balanced design satisfies $\text{Var}(\hat{\beta}_i) = \sigma^2/N$ with $N = 32R$ runs per dataset, the smallest level switch detectable at conventional power is $5.6 \sigma/\sqrt{N}$, and R is set so that this quantity falls below the threshold. Without this step a null result would be uninterpretable, since it would be consistent both with a genuinely absent effect and with a design too small to see one.

Whether an effect is confined to one task is assessed by comparing the per-dataset fits described in Section 4.4. A component whose coefficient changes sign or vanishes between the two datasets is task-dependent, and that conclusion is secure. A component whose coefficient agrees on both is not thereby shown to be task-independent in general, since two tasks cannot establish that; agreement is reported as the weaker statement that the effect is not peculiar to either of the tasks examined.

5.8 Validation

A model fitted on the data used to construct it has demonstrated association, not predictive content. Because the configuration space is enumerated exhaustively, there is by construction no unrun configuration to hold out, and the validation strategy of the preceding fractional study is therefore unavailable. Three substitutes are used instead.

Leave-one-cell-out prediction. The model is refitted 32 times, each time excluding all replicates of one configuration, and used to predict the excluded cell’s mean before that cell’s data are consulted. Aggregating over all 32 cells yields an out-of-sample coefficient of determination for each metric, which separates the metrics the design genuinely predicts from those it merely describes.

Comparison of the per-dataset fits. A model is fitted separately on each dataset and the two coefficient vectors are compared, component by component and with their confidence intervals. This is deliberately described as a comparison and not as a test. With two tasks, withholding one and predicting the other leaves a single fold in each direction, which is too little to estimate how an effect varies across tasks; a coefficient that agrees on both datasets is therefore evidence that the effect is not an artefact of one particular task, and nothing stronger. Disagreement is the more informative outcome of the two, since it establishes that the effect is task-dependent even though agreement could not have established the converse.

Audit of the fractional design. An eight-run Plackett–Burman subset of the 32 cells is extracted, a main-effects-only model is fitted to it exactly as in the preceding study, and its coefficients are compared with

those obtained from the full 32-configuration design. The discrepancy is a direct, empirical measurement of the error introduced by assuming that the components do not interact. This answers RQ3 and, unlike the other two checks, it produces a result that is informative for anyone designing such a study in future, whether or not they are able to afford full enumeration.

5.9 Controls and threats to validity

The following are held fixed across all conditions and are therefore not competing explanations for any measured effect: the agent model, pinned to a single dated snapshot rather than to a moving alias, so that the served weights cannot change while the study is running; the decoding temperature, held at one value in every run rather than varied with the replicate index; the train/validation partition and its seed; the executor implementation and its hyperparameter whitelist; the fixed portion of the template, which is byte-identical in all thirty-two instruction files; the response format; and the harness-level safety cap. Runs are mutually independent conversations, so no ordering or carry-over effect can arise between them.

The remaining source of variation is the sampling behaviour of the model itself, and it requires a more careful statement, because it is at once a threat to the measurement and a requirement of it. A model queried at a nonzero temperature does not return the same reply twice, and that variation could in principle be mistaken for an effect of the instruction file. It is controlled in three ways rather than removed. First, everything that is not the treatment is fixed, as listed above, so that a configuration and a sampling draw are the only quantities distinguishing one run from another. Second, the draw itself is made reproducible: every run is executed with an explicit sampling seed passed to the model interface, so that any individual run can be regenerated exactly, and the same list of seeds is used in every cell, so that the set of draws is identical across configurations and cannot favour one of them. Third, the design is balanced, so that whatever variation sampling introduces enters every cell equally and is absorbed into the error term rather than into a coefficient.

Removing the variation altogether, by decoding greedily at temperature zero, would be the wrong control in this design. The quantity against which every coefficient of equation (2) is tested is the run-to-run variance ε . A deterministic loop would drive that variance towards zero and with it render every standard error, every p -value and every equivalence bound meaningless, and an earlier iteration of this line of work encountered precisely that failure. Temperature is therefore held constant and strictly above zero. Fixing it converts it from an uncontrolled variable into a controlled constant of the study, while retaining the replicate-to-replicate variation on which the inference depends.

Two operational matters are settled by rule rather than after the fact. A run that terminates without producing a valid experiment is repeated with a fresh sampling seed until every cell contains exactly R completed runs, so that the balance on which the whole analysis rests is preserved by construction rather than assumed. And because an instruction is not invariably obeyed, the proportion of runs whose observed behaviour matches the instruction they were given is reported for each component, which is what distinguishes a component that had no effect from one that was not followed.

Three threats remain and are stated explicitly. The findings are conditional on one agent model; a different model may respond differently to the same instruction file, and the design measures the effect of instructions on one policy rather than a property of instructions in general. The five components are those identified as plausible levers by inspection of existing instruction files, and the study can say nothing about a component that was never varied. Finally, the task suite consists of tabular classification problems with a small, discrete hyperparameter space; whether the effect structure recovered here transfers to loops that search over code or over open-ended hypotheses is beyond what this design can establish.

6 Results and Discussion

Insert: What we have found in the experiments

7 Conclusion

Insert: contribution Insert: FUture work

References

- [1] T. Lütke. Performance: 53% faster parse+render, 61% fewer allocations. Pull request #2056, Shopify/liquid, 2026. Produced by an autoresearch run of approximately 120 experiments over 93 commits; unmerged at the time of writing. <https://github.com/Shopify/liquid/pull/2056>.
- [2] T. Lim, K. Muthuraman, and M. Sury. QRAFTI: An agentic framework for empirical research in quantitative finance, 2026. arXiv:2604.18500.
- [3] Shopify Engineering. Autoresearch isn’t just for training models, 15 April 2026. <https://shopify.engineering/autoresearch>.
- [4] B. Tychiev. A guide to Andrej Karpathy’s AutoResearch: automating ML with AI agents. DataCamp tutorial, 23 March 2026. Secondary source; it reports the contents of the repository’s program.md and results log. <https://www.datacamp.com/tutorial/guide-to-autoresearch>.
- [5] mutable-state-inc. autoresearch-at-home: experiment claiming, shared best-config syncing and hypothesis exchange across single-GPU agents. GitHub repository, 2026. <https://github.com/mutable-state-inc/autoresearch-at-home>.
- [6] MrTsepa. autoevolve: self-play evolution of code strategies with head-to-head evaluation, Elo / Bradley–Terry rating and Pareto-front branching. GitHub repository, 2026. <https://github.com/MrTsepa/autoevolve>.
- [7] trevin-creator. autoresearch-mlx: MLX-native Apple Silicon port preserving the upstream fixed-budget loop. GitHub repository, 2026. <https://github.com/trevin-creator/autoresearch-mlx>.
- [8] webfuse-com. awesome-autoresearch: a curated list of autonomous improvement loops, research agents and autoresearch-style systems. GitHub repository, 2026. <https://github.com/webfuse-com/awesome-autoresearch>.
- [9] tonitangpotato. autoresearch-engram: persistent cognitive memory with frequency-weighted retrieval of cross-session knowledge. GitHub repository, 2026. <https://github.com/tonitangpotato/autoresearch-engram>.
- [10] OSS Insight. 54,000 stars in 19 days: what karpathy/autoresearch tells us about the next frontier, 2026. <https://ossinsight.io/blog/autoresearch-overnight-ai-scientist>.
- [11] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes. Borg, Omega, and Kubernetes: lessons learned from three container-management systems over a decade. ACM Queue, 14(1):70–93, 2016. Also Communications of the ACM, 59(5):50–57, 2016.
- [12] The Kubernetes Authors. Controllers. Kubernetes documentation, concepts / cluster architecture. <https://kubernetes.io/docs/concepts/architecture/controller/>.

- [13] J. O. Kephart and D. M. Chess. The vision of autonomic computing. *IEEE Computer*, 36(1):41–50, 2003.
- [14] J. R. Boyd. *A Discourse on Winning and Losing*. Air University Press, Maxwell AFB, 2018. Collected briefings originally delivered from 1976 onwards.
- [15] G. Huntley. The Ralph technique: running a coding agent in an autonomous shell loop with a fresh context window per iteration, 2025. Practitioner write-up; not peer reviewed. **[TODO: insert the exact post URL and access date.]**
- [16] F. P. B. Osinga. *Science, Strategy and War: The Strategic Theory of John Boyd*. Routledge, Abingdon, 2007.
- [17] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024. arXiv:2405.15793.
- [18] J. Bergstra and Y. Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.
- [19] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes. Autonomous chemical research with large language models. *Nature*, 624(7992):570–578, 2023.
- [20] X. Bouthillier et al. Accounting for variance in machine learning benchmarks. *Proceedings of Machine Learning and Systems*, 3:747–769, 2021.
- [21] A. Feder, K. A. Keith, E. Manzoor, R. Pryzant, D. Sridhar, Z. Wood-Doughty, J. Eisenstein, J. Grimmer, R. Reichart, M. E. Roberts, B. M. Stewart, V. Veitch, and D. Yang. Causal inference in natural language processing: Estimation, prediction, interpretation and beyond. *Transactions of the Association for Computational Linguistics*, 10:1138–1158, 2022. Preprint arXiv:2109.00725, 2021.
- [22] Q. Huang, J. Vora, P. Liang, and J. Leskovec. MAgentBench: Evaluating language agents on machine learning experimentation. *ICML*, 2024. arXiv:2310.03302.
- [23] F. Hutter, L. Kotthoff, and J. Vanschoren, editors. *Automated Machine Learning: Methods, Systems, Challenges*. Springer, 2019.
- [24] A. Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically. GitHub repository, 2026. <https://github.com/karpathy/autoresearch>.
- [25] R. D. King, J. Rowland, S. G. Oliver, M. Young, W. Aubrey, E. Byrne, M. Liakata, M. Markham, P. Pir, L. N. Soldatova, A. Sparkes, K. E. Whelan, and A. Clare. The automation of science. *Science*, 324(5923):85–89, 2009.
- [26] P. Liang, R. Bommasani, T. Lee, et al. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023. arXiv:2211.09110.
- [27] Z. C. Lipton and J. Steinhardt. Troubling trends in machine learning scholarship. *ACM Queue*, 17(1):45–77, 2019. Preprint arXiv:1807.03341, 2018.
- [28] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha. The AI Scientist: Towards fully automated open-ended scientific discovery, 2024. arXiv:2408.06292.
- [29] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr. Quantifying language models’ sensitivity to spurious features in prompt design, or: How I learned to start worrying about prompt formatting. *ICLR*, 2024. arXiv:2310.11324.
- [30] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. *NeurIPS*, 2023. arXiv:2303.11366.

- [31] A. Webson and E. Pavlick. Do prompt-based models really understand the meaning of their prompts? NAACL-HLT, pages 2300–2344, 2022.
- [32] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. ICLR, 2023. arXiv:2210.03629.
- [33] Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, and J. Ba. Large language models are human-level prompt engineers. ICLR, 2023. arXiv:2211.01910.
- [34] C. Fawcett and H. H. Hoos. Analysing differences between algorithm configurations through ablation. *Journal of Heuristics*, 22(4):431–458, 2016.
- [35] F. Hutter, H. Hoos, and K. Leyton-Brown. An efficient approach for assessing hyperparameter importance. ICML, 2014.
- [36] R. L. Plackett and J. P. Burman. The design of optimum multifactorial experiments. *Biometrika*, 33(4):305–325, 1946.